

Programming in C

Lab 4: Loops

Lecturer: *Dr. Wan-Lei Zhao*
Spring Semester 2022

Approximate exp(x) (1)

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!} = 1 + \frac{x}{1} + \frac{x^2}{2} + \frac{x^3}{6} \dots \quad (1)$$

- Requirements
 - Keep terms that larger than 1e-6
- Hints
 - Define $\text{term} = \frac{x^n}{n!}$
 - Do loop while `abs(term)` is larger than 1e-6

Approximate exp(x) (2)

```
1 #include <stdio.h>
2 #include <math.h>
3 int main()
4 {
5     const double prec = 1e-6;
6     double term = 1, sum = 0, x = 0.3;
7     double up = 1, low = 1;
8     int i = 1;
9     scanf("%lf", &x);
10    while(abs(term) > prec)
11    {
12        sum += term;
13        up = up*x;
14        low = low*i;
15        term = up/low;
16        i++;
17    }
18    printf("%lf\n", sum);
19 }
```

Print pyramid of alphabets on the screen

```
A
BBB
CCCCC
DDDDDDD
EEEEEEEEEE
```

- Hints
 - Suggested to use **for** loop
 - Two levels of embedding

```
1 #include <stdio.h>
2 for(i = 0; i < 5; i++){
3 {
4     for(j = 0; j < ?; j++){
5     {
6         //filling your code
7     }
8 }
```

Answer

```
1 #include <stdio.h>
2 int main()
3 {
4     int i=0, j=0, count=0;
5     int nl=5, nc=1, nb=nl-1;
6     for(j = 0; j < nl; j++)
7     {
8         for(i = 0; i < nb; i++)
9         {
10             printf(" ");
11         }
12         nc = 2*j-1;
```

```
13         for(i=0;i<nc;i++)
14         {
15             printf("%c", ch);
16         }
17         ch++;
18         nb--;
19         printf("\n");
20     } //for(j)
21 }
```

Find out prime numbers

- Requirements
 - Find all prime numbers smaller than **500**
 - Print out 8 numbers on each line
- Hints
 - Only dividable by itself
 - By filtering method
 - Try “%” from **2** to **sqrt(num)**

Answer (1)

```
1 #include <stdio.h>
2 #include <math.h>
3
4 int main()
5 {
6     int _PRIME_ = 1;
7     float b = 0;
8     int i = 0, j = 0, count = 0;
9     for(i = 2; i <= 500; i++)
10     {
11         b = sqrt(i+0.0);
12         _PRIME_ = 1;
13         for(j = 2; j < b; j++)
14         {
15             if(i%j == 0 && i != 2)
16             {
17                 _PRIME_ = 0;
```

Answer (2)

```
18         } // if (i%j)
19     } // for(j)
20     if (_PRIME_ == 1)
21     {
22         count++;
23         printf("%d\t", i);
24         if (count%8 == 0)
25         {
26             printf("\n");
27         }
28     } // if
29 } // for(i)
30 if (count %8 !=0)
31     printf("\n");
32 }
```

Find out all complete numbers

- Find out all the **complete number** in the range of [1, 10000]
- Complete number: it equals to the sum of its factors
- Example: $6 = 1+2+3$

Answer

```
1 #include <stdio.h>
2 int main()
3 {
4     int j=0, i=0, sum=0;
5     for(j=1; j<=10000; j++)
6     {
7         sum = 1;
8         for(i = 2; i < j; i++)
9         {
10             if(j%i == 0)
11             {
12                 sum += i;
13             }
14         } // for(i)
15     }
```

```
16         if(sum == j)
17         {
18             printf("%d\t", j);
19         }
20     } // for(j)
21     printf("\n");
22 }
```

Convert pure decimal fraction into binary form

- Convert pure decimal fraction such as '0.635' into its binary form '0.1010001010001111010111'
- Accept a pure decimal fraction from input: 0.625
- Output its binary form: 0.101
- The loop continues until the fraction is lower than 0.005

Answer

```
1 #include <stdio.h>
2 int main()
3 {
4     float a = 0.635;
5     scanf("%f", &a);
6     if(a < 1.0){
7         printf("0.");
8         do{
9             a = a*2;
10            if(a >= 1.0)
11            {
12                printf("1");
13                a = a - 1.0;
14            }else{
15                printf("0");
16            }
17        }while(a > 0.005);
18    }//end if
19    return 0;
20 }
```